

5.高性能的秘密：图层与合成

一、渲染流程的回顾

在我们深入探讨图层之前，让我们再次回顾一下简化的渲染流程：

JavaScript → Style → Layout (布局) → Paint (绘制) → Composite (合成)

大多数CSS属性的更改（例如 `width`，`height`，`left`，`top`）都会触发 **Layout** 和 **Paint**，这被称为 **回流(Reflow)** 和 **重绘(Repaint)**，是浏览器中非常耗费计算资源的操作。如果动画的每一帧都经历这个过程，就很容易导致卡顿。

二、图层 (Layer)

为了优化这个过程，浏览器引入了**图层 (Layer)**的概念。你可以把网页想象成一个Photoshop或Sketch文件，它不是一个单一的平面，而是由多个堆叠在一起的图层组成的。

- **默认情况:** 在默认情况下，页面上的所有元素都位于同一个默认图层中。
- **提升为独立图层:** 在某些特定情况下，浏览器会为某个元素创建一个新的、独立的图层。这个过程被称为“层提升” (Layer Promotion)。

常见的触发层提升的CSS属性和HTML元素包括：

- 3D 或透视变换属性：`transform: translate3d(x,y,z)`，`rotate3d(...)`，`perspective(...)` 等。
- 使用了 `will-change` 属性（这是一个专门用于性能优化的属性，可以提前告知浏览器该元素将要发生变化）。
- `<video>` 和 `<canvas>` 元素。
- 具有 `position: fixed` 的元素。
- 一些复杂的动画，例如 `opacity` 和 `transform` 的动画。

当一个元素被提升到独立的图层后，它就拥有了自己的“位图” (Bitmap)，可以独立于页面的其他部分进行变换和绘制。

三、合成 (Compositing)

有了分离的图层后，渲染流程的最后一步就是**合成 (Compositing)**。

合成是指浏览器将所有独立的图层按照正确的顺序和位置组合、叠加，最终生成一个完整的屏幕图像的过程。

关键在于，这个合成工作可以被 **GPU (图形处理器)** 高效地执行。GPU专门为图形计算而设计，处理位图的移动、缩放、旋转和透明度变化等操作速度极快。

四、transform 和 opacity 高性能的秘密

现在，我们可以揭示 transform 和 opacity 实现高性能动画的秘密了：

1. **跳过布局和绘制:** 当你为一个被提升到独立图层的元素应用 transform (移动、缩放、旋转) 或 opacity (改变透明度) 动画时，浏览器能够识别出这个变化不会影响到其他元素的布局 (Layout)，因此可以完全跳过 Layout 和 Paint 这两个最耗时的阶段。
2. **GPU 加速:** 浏览器只需将这个已经绘制好的图层（作为一个位图）发送给 GPU。然后，在动画的每一帧，CPU 都不需要做任何事情，只需通知 GPU 去执行这个图层的变换（例如，将其向右移动几个像素）或改变其透明度即可。

这个过程的渲染路径被大大缩短：

JavaScript → Style → Composite

由于整个过程由高度专业化的 GPU 直接处理，动画可以非常流畅，轻松达到 60fps (每秒60帧)，用户感知不到任何卡顿。这也就是我们常说的“GPU加速”。

五、结论与最佳实践

- **优先使用 transform 和 opacity:** 在实现位移动画、缩放、旋转或淡入淡出效果时，应始终优先使用 transform 和 opacity 属性，而不是去更改 top, left, margin 或 width, height。
- **合理利用 will-change:** 如果你知道一个元素即将进行动画，可以使用 will-change: transform, opacity; 来提前“暗示”浏览器，让其为该元素创建独立图层，做好优化准备。
- **避免滥用图层:** 虽然图层可以提升性能，但创建过多的图层会占用大量内存和GPU资源，反而可能导致性能下降。因此，应有节制地使用，避免不必要的层提升。